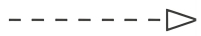
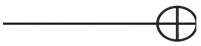
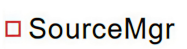
COL740 Assignment-1

PV Revanth (2022CS51650), Sarthak Panda (2022CS51217)

1 Introduction

As part of this assignment, our task was to create an automated pipeline for generating UML diagrams corresponding to important sections of codebases. We restrict our analysis to C++ repositories. The main tools used for this purpose are **clang-uml** and **plant-uml**. To identify the most important parts of a codebase, we apply heuristics and standard algorithms such as **Degree Centrality** and **PageRank** to select the top-K classes. These classes represent the most significant sections of the repository and are displayed in the UML. More details about the top-K class selection are discussed in a later section.

2 Understanding the Arrows in PlantUML Diagrams

Arrow	Description
	This Arrow in the PlantUML usually denotes “is-a-instance-of” relation, where Object at Arrow tail is a instance of Class at Arrow Head.
	This Arrow in the PlantUML indicates inheritance, The Arrow head points to the parent class and the tail points to the derived class.
	This Arrow is usually shown in PlantUML’s diagram when Tail Class’s Function (or Templated Class at Tail) uses Head class object as an Argument.
	This arrow in PlantUML diagrams represents an aggregation relationship (i.e., no ownership). It means that the class at the tail has a reference to an instance of the class at the head—typically through a shared_ptr or weak_ptr. This allows the lifetime of the head class instance to be independent of the aggregator (tail) class.
	This arrow in PlantUML diagrams typically represents a composition relationship (i.e., ownership). It means that the class at the head contains an object (or a strong reference, usually through a unique_ptr) to the class at the tail. This means the lifetime of the tail class instance is tied to the lifetime of the head class.
	This anchor arrow in PlantUML marks that the element at the tail end is owned by / declared in the namespace of the class it points to, i.e. nesting / membership (e.g. A nested class/enum inside its declaring class)
	These details are usually observed along with the aggregation or composition arrows described above. They indicate the name of the object involved in the relationship, while the red square box beside it shows that the object is a private member of the class it belongs to.
	Similar to above, only difference is that yellow diamond besides the object shows that it is a protected member of the class it belongs to.
	Similar to above, only difference is that green circle besides the object shows that it is a public member of the class it belongs to.

3 Requirements & Setup

The below presented pipeline requires Ubuntu 22.04 (or compatible Linux distribution) with several other essential packages and tools. The following installation commands will prepare the system to run the complete UML diagram generation and analysis pipeline. (You may take help from the Makefile for the setup described here; See Section 5 for details).

```
sudo apt update && sudo apt upgrade -y
# Install essential build tools and development libraries
sudo apt install -y git build-essential cmake pkg-config
# Install C/C++ compilation tools
sudo apt install -y gcc g++ make
# Install Python 3 and pip
sudo apt install -y python3 python3-pip
# Install Java Runtime Environment (required for PlantUML)
sudo apt install -y default-jre
# Install GraphViz (required for PlantUML diagram rendering)
sudo apt install -y graphviz
# Install PlantUML
sudo apt install -y plantuml
# Install clang-uml from PPA
sudo add-apt-repository ppa:bkryza/clang-uml
sudo apt update
sudo apt install clang-uml
```

For detailed installation instructions and troubleshooting related to clang-uml, refer to the official clang-uml documentation: `https://github.com/bkryza/clang-uml/blob/master/docs/installation.md`

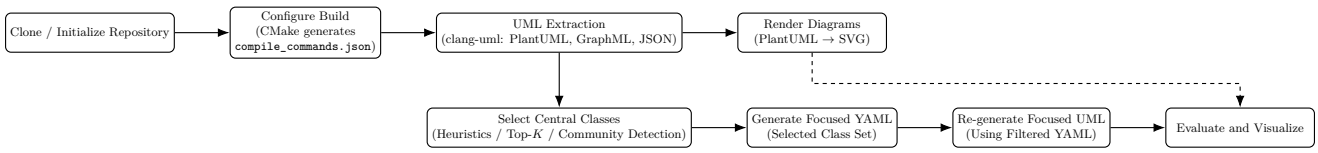
4 Detailed Approach For Automated UML Generation

We tested our approach on several sufficiently large C++ repositories that are publicly available. The repositories used in our experiments are:

- **`nlohmann/json`** *(JSON for Modern C++)*
- **`zeux/pugixml`**
- **`libcpr/cpr`** *(C++ Requests)*

Although we will explain the **UML generation procedure** using `nlohmann::json` as a worked example, the steps are described in a generalized way so they can be applied to any C++ repository. In a later section we discuss the results we obtained from the repositories listed above.

4.1 Overall Pipeline



Overall Pipeline

The above figure shows the overall strategy/approach that we are using. We will understand these steps one by one. For now let us focus on the first step.

4.1.1 Step-1: Configure build

This is the only step where some manual effort might be needed is preparing a `CMakeLists.txt` (if the repository does not already include one). Below we explain what a `CMakeLists.txt` represents, when you need to create it, and how to create a minimal one.

What is `CMakeLists.txt` and why it matters

- `CMakeLists.txt` is the input file used by CMake to describe how a project is built i.e. 'which sources to compile', 'include directories', 'compile options', 'targets' (libraries/executables), and 'dependencies'.
- Many clang-based UML extractors (and tools that consume `compile_commands.json`) rely on accurate compile flags and include paths to parse the sources correctly. Generating `compile_commands.json` (via CMake with `CMAKE_EXPORT_COMPILE_COMMANDS=1`) is therefore the most common way to give clang-uml the information it needs.

When we might need to create `CMakeLists.txt` manually

- **The repo uses a different build system (Makefiles, Bazel, custom scripts)**: In that case we may need to either (a) run an intercept tool (e.g. `intercept-build`) while building to create `compile_commands.json`, or (b) write a minimal `CMakeLists.txt` to describe the build for the files of interest.
- **The repo is header-only**: CMake often won't produce compile commands for headers by default. A workaround is to add a small "dummy" translation unit (`main.cpp`) that includes the project's headers so the compile database contains the necessary include/flag entries.

Let us now look at examples CMakeLists.txt creation for the above described two case.

- **The repo is header-only**: In the case of header-only libraries (for example, `nlohmann::json`) the project may already include a `CMakeLists.txt` only for tests or examples. Since the repo is header only library one can directly use that or another simple way is to create a tiny "dummy" translation unit (e.g., `main.cpp`) that includes a library header (or a small wrapper header that includes the library's public entry points). Building this dummy target causes CMake to record the include paths and compile flags in the compile database, which clang-based tools (like `clang-uml`) can then consume.

```
#include "include/nlohmann/json.hpp"
int main() {
    return 0;
}

```

Listing 1: main.cpp

```
cmake_minimum_required(VERSION 3.10)
project(json_analysis)

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

add_executable(dummy main.cpp)
target_include_directories(dummy PUBLIC "${CMAKE_CURRENT_SOURCE_DIR}/include")

```

Listing 2: CMakeLists.txt

- **The repo uses a different build system** : For repositories that do not use CMake (for example, Makefile-based or custom build systems), the practical way to obtain a `compile_commands.json` is to Intercept the real build. We can use `bear` or `intercept-build` like tools to capture the compiler invocations while the project is built. This method is non-intrusive and typically produces a complete `compile_commands.json`.

```
# Install bear:
sudo apt install bear

bear -- make -j4

# OR using intercept-build (clang's toolchain)
intercept-build make -j4

# After the build the compile_commands.json will be generated in the repo root
ls -l compile_commands.json

```

Listing 3: Capture compile commands using `bear` or `intercept-build`

4.1.2 Step 2 : UML Extraction

Note: You may optionally skip this section if you opt to use the automated script described later, it is kept here for the sake of completeness.

First of all let us write a minimal general clang configuration file.

```yaml
# .clang-uml
compilation_database_dir: build
output_directory: docs/diagrams
generate_method_arguments: none
diagrams:
  all_classes:
    type: class
```

Listing 4: Minimal `.clang-uml` configuration

Also make sure following directory structure is followed.

Repo
```
├> CMakeLists.txt
├> .clang-uml
├> Heuristics.py
├> (Other Repo Files)
```

Now we can run the commands below to generate a **class UML diagram** for the entire repository. The UML produced this way is a direct extraction (i.e., *naive*) and can be very large, making it difficult to read or interpret. In the next sections we discuss several pruning techniques (for example: selecting top-k central classes, grouping by community, and truncating long class members) that produce much smaller, more useful diagrams.

```bash
cd Repo
# 1. Configure the build system:
#    Generate a compile_commands.json file in Repo/build that clang-based tools use.
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=1
# 2. Generate UML diagrams (PlantUML, GraphML, JSON) using the .clang-uml config:
clang-uml -c .clang-uml -g plantuml -g graphml -g json
# 3. Convert the generated PlantUML file to an SVG image:
plantuml -tsvg docs/diagrams/all_classes.puml
```

Listing 5: Generate UML and render SVG

4.1.3 Step 3 : Detailed Algorithm and Heuristics for pruning the class UML diagram

The Python script performing Heuristic Pruning expects a `GraphML` file produced by clang-uml as its input. It first parses the GraphML to extract node IDs, fully qualified names, and node types (e.g.namespace, type), and then loads the graph structure itself with `networkx.read_graphml` into an intermediate NetworkX graph G. Before proceeding, we remove any self-loops and collapse multi-edges (i.e. only one edge is kept per node pair). The main output is a clang-uml-compatible YAML that lists the selected fully qualified element names i.e. the output YAML contains an `include.elements` array holding the chosen top-K classes from the heuristic as `namespace::ClassName` entries, so the YAML can be fed back to clang-uml to generate focused diagrams. Let us Look at Heuristic's High Level Idea before moving onto Heuristic's Pseudo Code.

Heuristic's High Level Idea

The algorithm first computes a broad set of per-node centrality and structural metrics (*degree*, *PageRank*, *betweenness*, *eigenvector*, *closeness*, *k-core*, *neighbor-h*, *articulation status*, and a *manifold-ranking score*). These metrics are normalized and collapsed into a single *importance score* per node (either via a tunable weighted sum or an unweighted average). Rather than simply returning the global top-k nodes by that score, the selection is **community-aware** i.e. the algorithm detects graph communities using greedy modularity maximization, ranks communities by the sum of their members' importance scores, and then selects the top-p nodes from the highest-scoring communities until k nodes have been collected. If community-aware selection does not yield enough nodes, the algorithm fills the remainder with the globally top-scoring nodes. This **community-boosted** heuristic balances choosing globally important nodes with ensuring coverage across structurally coherent subgraphs.

Understanding the Graph Metrics used in Heuristic

Centrality and structural metrics	
Metric	**Formula for generic node n**
Degree centrality	$\text{score}(n) = \deg(n)$ (normalized: $\deg(n)/(N-1)$).
PageRank	$\text{score}(n) = \dfrac{1-d}{N} + d \sum_{m \in \text{In}(n)} \dfrac{\text{score}(m)}{\text{outdeg}(m)}$, iterated to convergence; $d \in (0,1)$.
Betweenness centrality	$\text{score}(n) = \sum_{s \neq n \neq t} \dfrac{\sigma_{st}(n)}{\sigma_{st}}$, where $\sigma_{st} = \#$ shortest paths $s \to t$ and $\sigma_{st}(n) = \#$ of those through n. (Normalize for undirected graphs by dividing by $(N-1)(N-2)/2$.)
Eigenvector centrality	$\text{score}(n) = \dfrac{1}{\lambda} \sum_m A_{nm}\, \text{score}(m)$, i.e. $\mathbf{s}$ is the principal eigenvector of adjacency matrix A: $A\mathbf{s} = \lambda\mathbf{s}$.
Closeness centrality	$\text{score}(n) = \dfrac{N-1}{\sum_{m \neq n} d(n,m)}$, here $d(n,m)$ is the shortest path distance (number of steps or sum of weights) between node n and node m.
Core number (k-core)	$\text{score}(n) = \max\{k \mid n \in k\text{-core}(G)\}$, i.e. largest k such that n remains after iteratively removing nodes with degree $< k$.
H-index (node)	$\text{score}(n) = \max\{h \mid \text{at least } h \text{ neighbors of } n \text{ have degree} \geq h\}$. (Also sometimes defined using neighbors' centralities instead of degree.)
Articulation point indicator	$\text{score}(n) = \begin{cases} 1 & \text{if } \kappa(G \setminus \{n\}) > \kappa(G) \\ 0 & \text{otherwise} \end{cases}$ where $\kappa(\cdot)$ is the number of connected components.
Manifold ranking	Let S be a (row/column)-normalized similarity matrix, $\mathbf{y}$ initial labels/scores and $\alpha \in (0,1)$. Then $\mathbf{f} = \alpha S\mathbf{f} + (1-\alpha)\mathbf{y} \quad \Rightarrow \quad \mathbf{f} = (1-\alpha)(I - \alpha S)^{-1}\mathbf{y}$, so $\text{score}(n) = f_n$.

See the NetworkX centrality reference for more details: NetworkX-Centrality (reference).

Heuristic's Pseudo Code

```
procedure TopK_Select(G, k, p, filter_ns)
    // Compute metrics for every node n ∈ G
    Compute centrality and structural metrics: degree, pagerank, betweenness, eigenvector, closeness, core,
    h-index, articulation, manifold
    // Normalize each metric (min–max) to [0,1]
    Normalize metrics {M_i(n)}
    // Produce a single score per node (average or weighted sum)
    score(n) ← (1/|M|) Σ_{i∈M} M_i(n)
    // Community detection
    C ← greedy_modularity_communities(G)
    // Score communities by member scores
    for all C ∈ C do
        comm_score(C) ← Σ_{n∈C} score(n)
    end for
```

```
    Sort 𝒞 by comm_score descending
    S ← [ ]                                                    ▷ selected nodes
    for all C in sorted 𝒞 do
        L ← members of C sorted by score(·) descending
        p ← 0
        for all n in L do
            if filter_ns and is_namespace(n) then continue
            end if
            append n to S
            p ← p + 1
            if p ≥ per_community then break
            end if
        end for
        if |S| ≥ k then break
        end if
    end for
    if |S| < k then
        F ← all nodes sorted by score(·) descending
        for all f in F do
            if f ∉ S then
                if filter_ns and is_namespace(f) then continue
                end if
                append f to S
                if |S| ≥ k then break
                end if
            end if
        end for
    end if
    return first k items of S
end procedure
```

5 Combining all the steps to User-Friendly Makefile

Makefile Command Reference

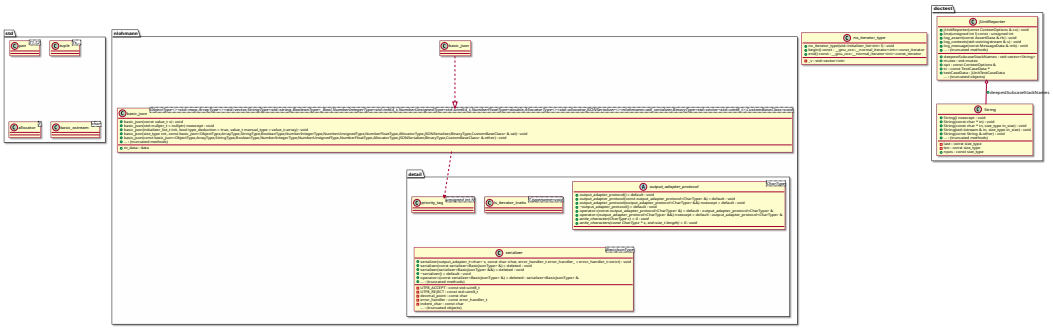
Command	Description
`make setup`	Performs initial system setup i.e. updates package manager, installs system dependencies (like git, build-essential, plantuml, clang-uml, and all other described in Section 3 , creates Python virtual environment in `venv/`, and installs Python packages from `requirements.txt`.
`make install`	Ensures virtual environment exists and installs/updates Python dependencies from `requirements.txt`. Lighter than `setup` & only handles Python environment without system packages.
`make run REPO=`*`name/path-to-repo-root`*	Main pipeline for C++ repository analysis. First the Makefile verifies that the repository root path exists and is accessible. Then it validates the CMake project structure, runs `clang-uml` to create class diagrams, executes `boosted_community_rep.py` for uml pruning (top-15 classes) then runs `uml_beautifier.py` to produce final refined diagrams, and writes `initial.svg` and `final.svg` to `output/`*`name`*`/`.
`make clean`	Removes virtual environment directory (`venv/`). Optionally cleans repository build artifacts and output diagrams when `REPO` or `REPO_ROOT` variables are specified (e.g., `make clean REPO=json_my`).

Note: the Makefile is provided for ease of use by the end user, but it only works for pure C++ libraries that already include a working CMakeLists.txt, otherwise you will need to create one manually (see Section 4.1.1 for guidance). More importantly, make sure you install the target repo's dependencies for example, building `libcpr/cpr` requires Meson and Ninja, without those, the CMake configuration or build will fail and the automated script provided here may not work.

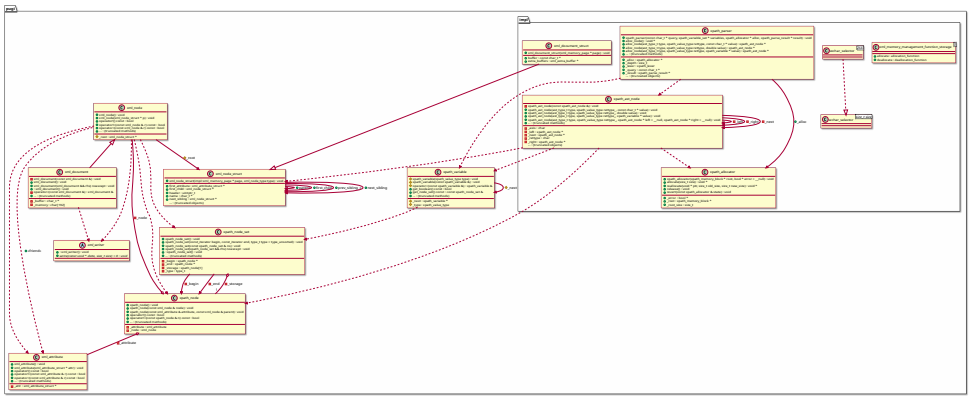
6 Results

As earlier mentioned We ran our automated script on three repositories, We will be keeping those results below.

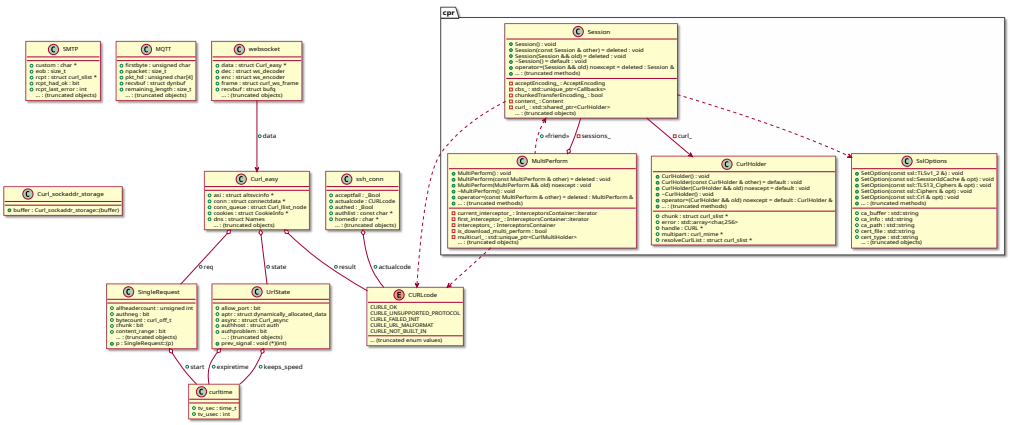
6.1 nlohmann/json



6.2 zeux/pugixml



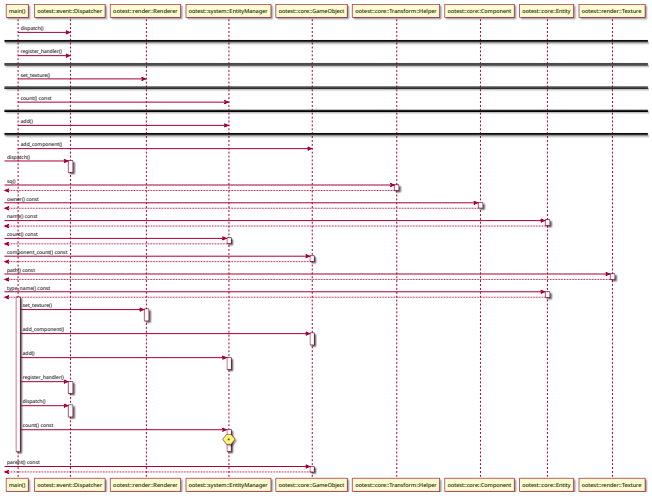
6.3 libcpr/cpr



The UML diagrams show the top 15 classes (selected by a heuristic) in each of the three repositories. Note: some items chosen by our strategy are actually nested structs & enum and therefore don't render as objects in the

diagrams with our strategy (a small caveat, you may refer to Readme.md for more details in this regard), so a few diagrams may contain slightly fewer than 15 classes. SVGs are provided hence you may zoom in for details.

7 Small note on Sequence Diagram UML



We keep the sequence-diagram output produced while testing a small dummy repository. This diagram was also generated with `clang-uml`. We did not find a fully automated way to prune the diagram, one viable approach is to automatically select the `from` and `to` (start/end) function calls, for example, by choosing the longest call path or the most frequently used call path. For now, the `from` and `to` points are chosen manually in the configuration file.

8 Submission Details

We have provided **six files** and one **dummy repository directory** to help user to get started and one resource folder for Sequence UML Diagrams. Place your custom repository root in the working directory (illustrated below), then run the `make` commands described in Section 5.

Working Directory

```
├ > Your Repo Root
├ > oo-test (Dummy Repo)
├ > Sequence-Diagram-UML-Resouces
├ > boosted_community_rep.py
├ > uml_beautifier.py
├ > requirements.txt
├ > Makefile
├ > Readme.md
├ > report.pdf
```

Example:

```
# initialize environment
make setup

# run the pipeline against your repo placed in the working directory
make run REPO=./oo-test
```

End of Report